Blockchain Exp - 4

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory:

1. **Primitive Data Types, Variables, Functions – pure, view**

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int**: unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool**: represents logical values (true or false).
- **address**: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string**: store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure**: cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view**: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. **Inputs and Outputs to Functions**

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
    - public: available both inside and outside the contract.
    - private: only accessible within the same contract.
    - internal: accessible within the contract and its child contracts.
    - external: can be called only by external accounts or other contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

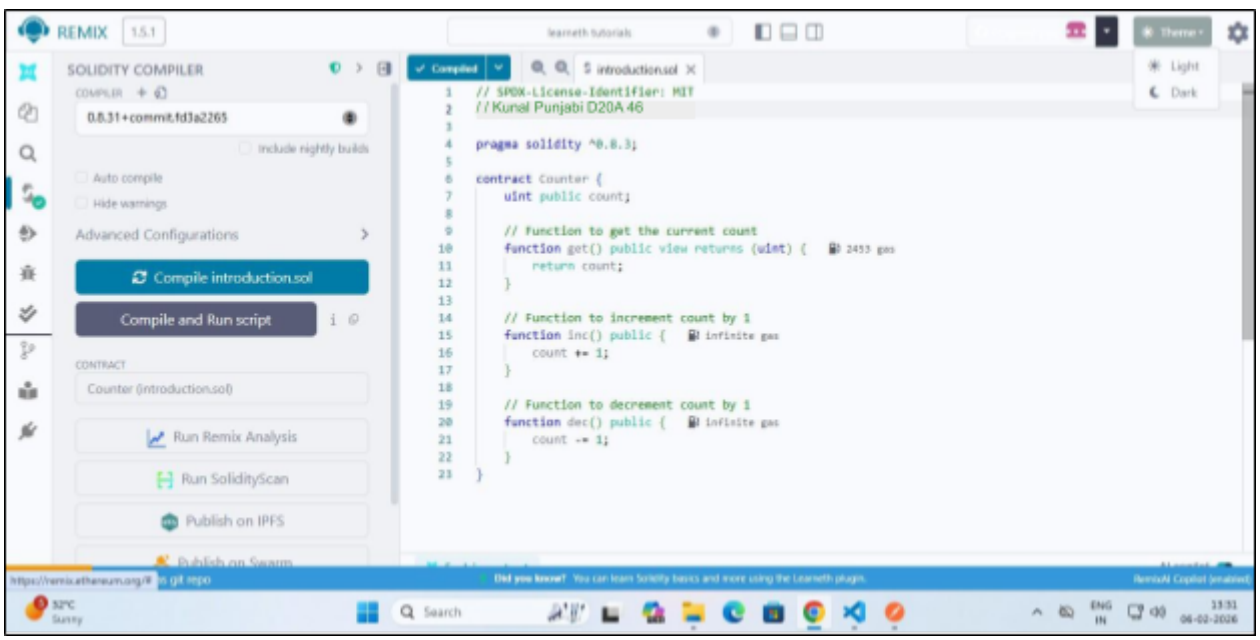
- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

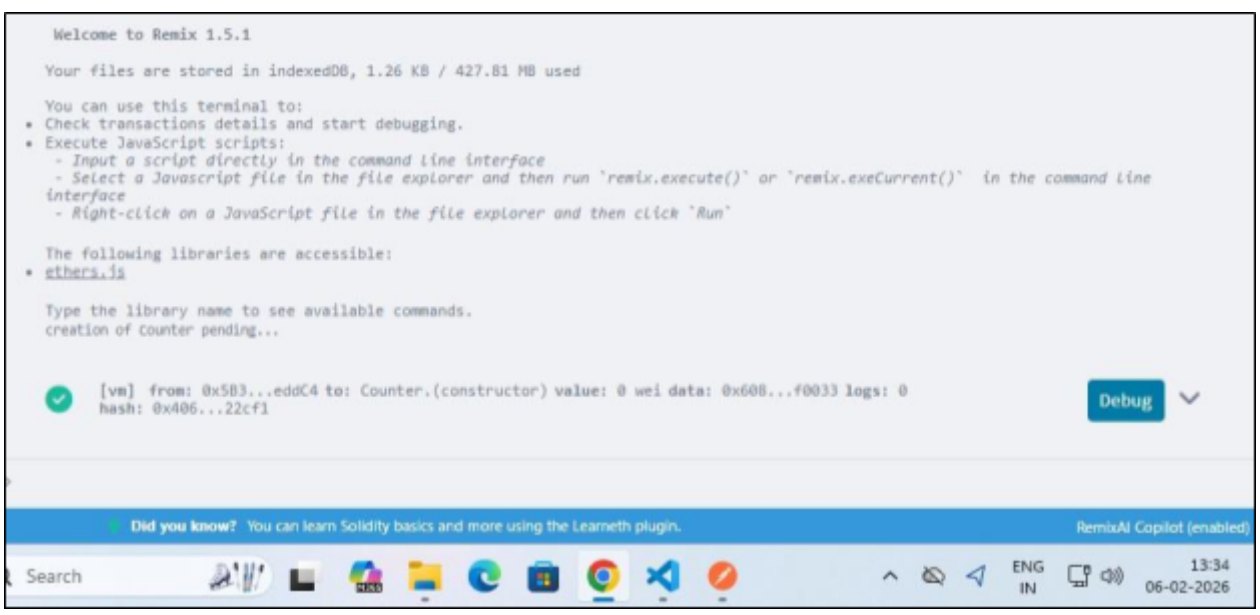
7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

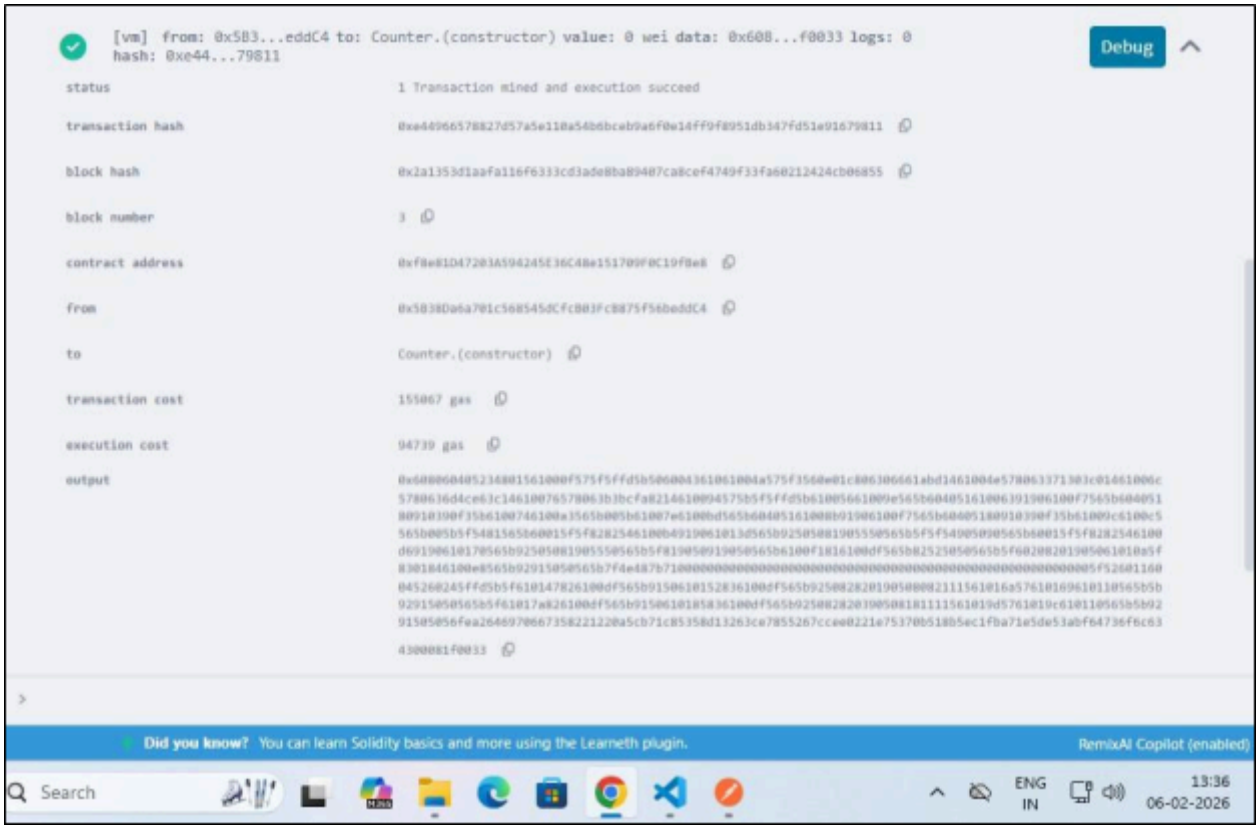
- **Ether and Wei**: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price**: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions**: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

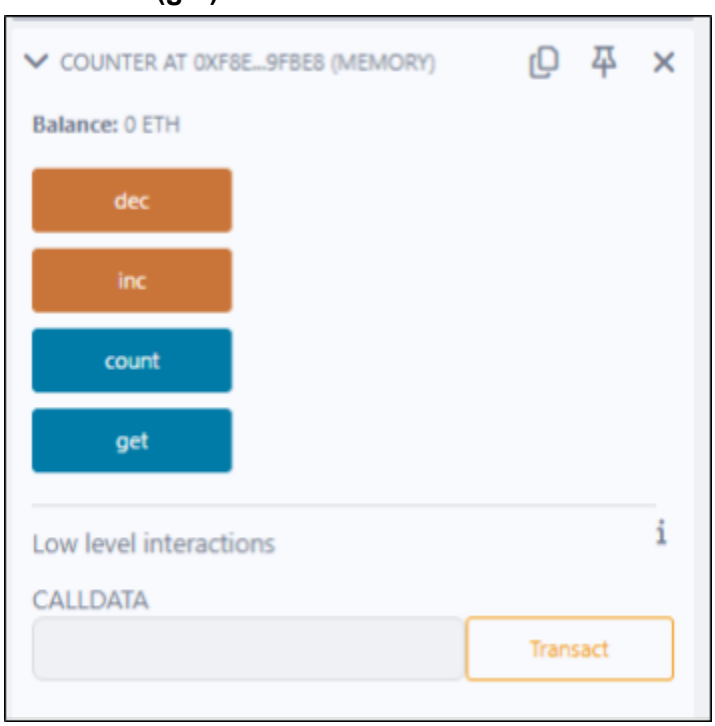
Tutorial - 1 (compile)



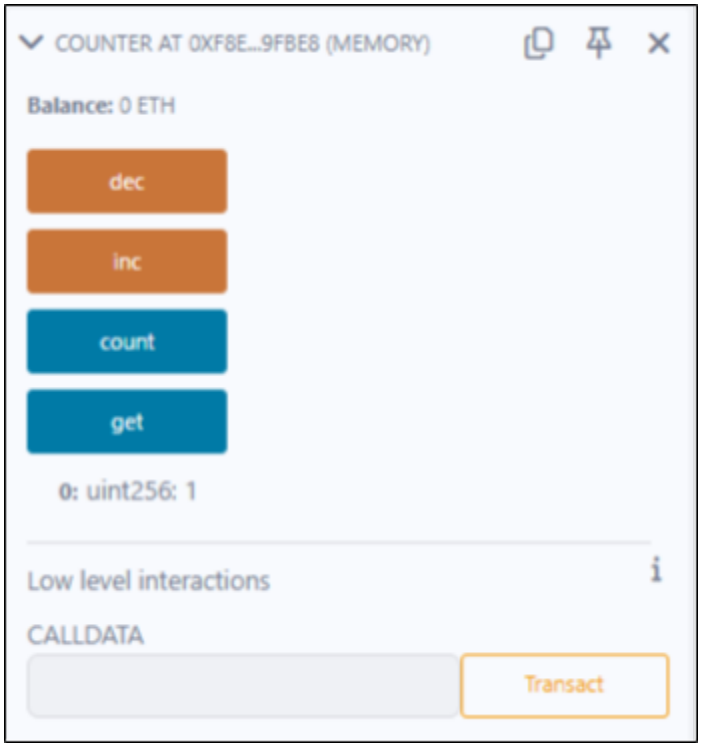




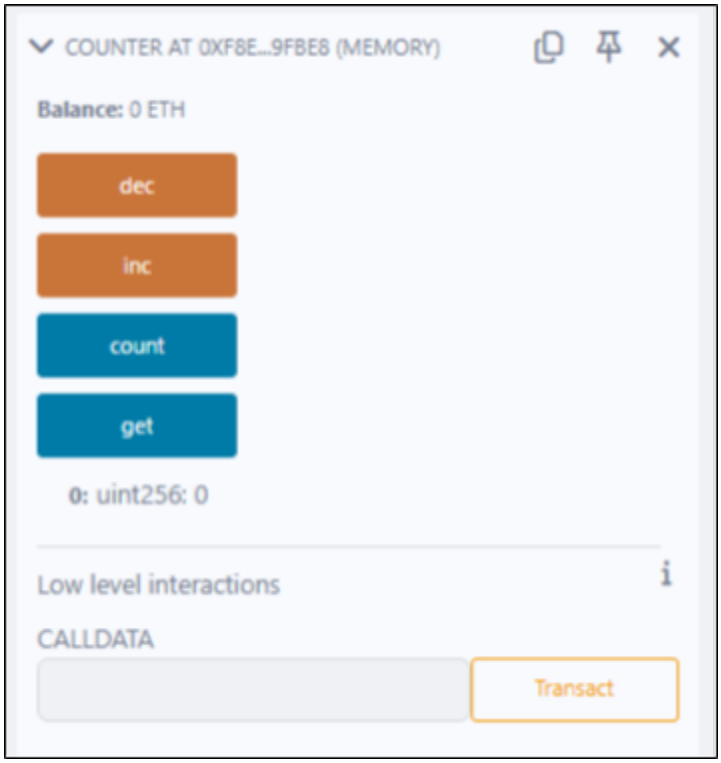
Tutorial - 1 (get)



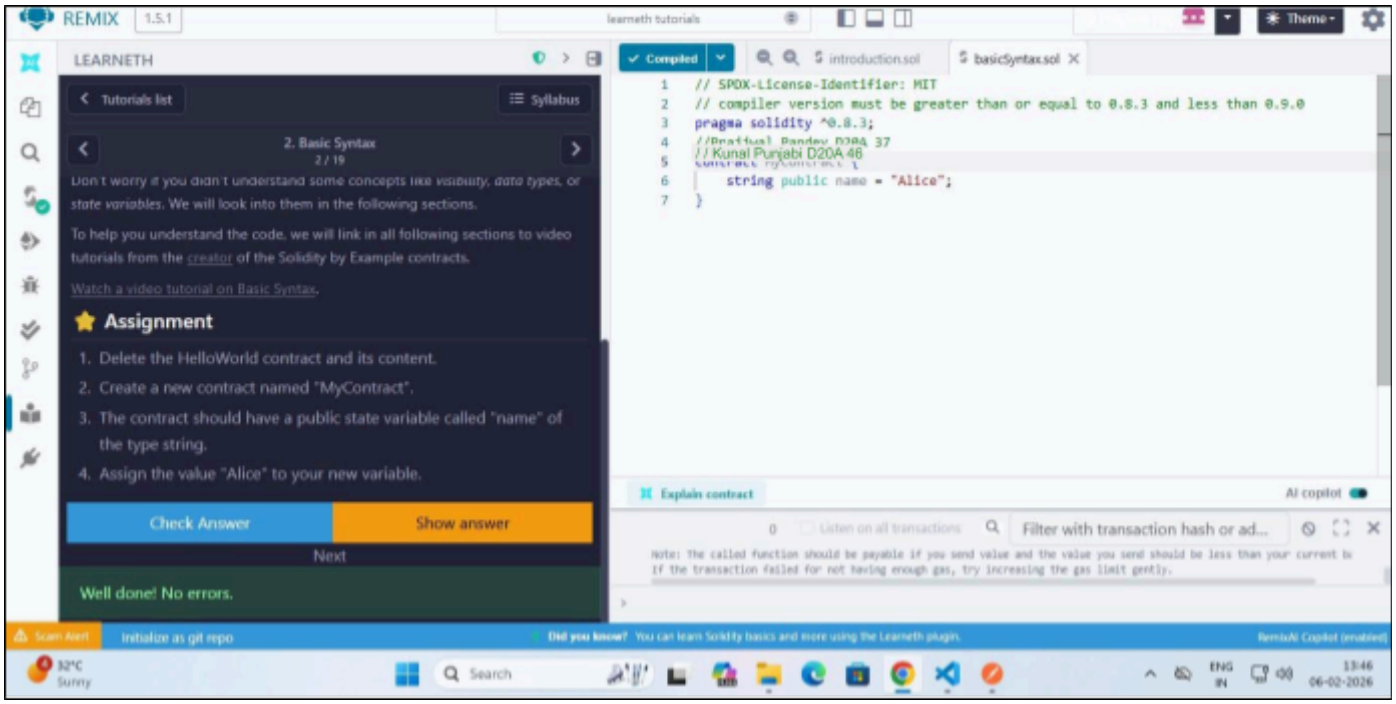
Tutorial - 1 (inc)



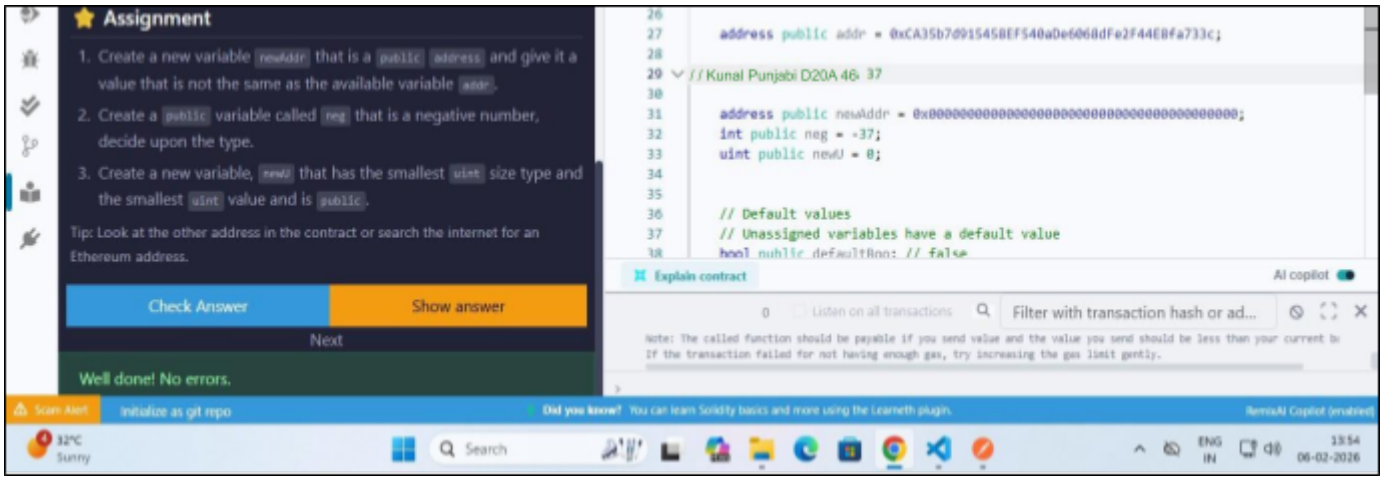
Tutorial - 1 (dec)



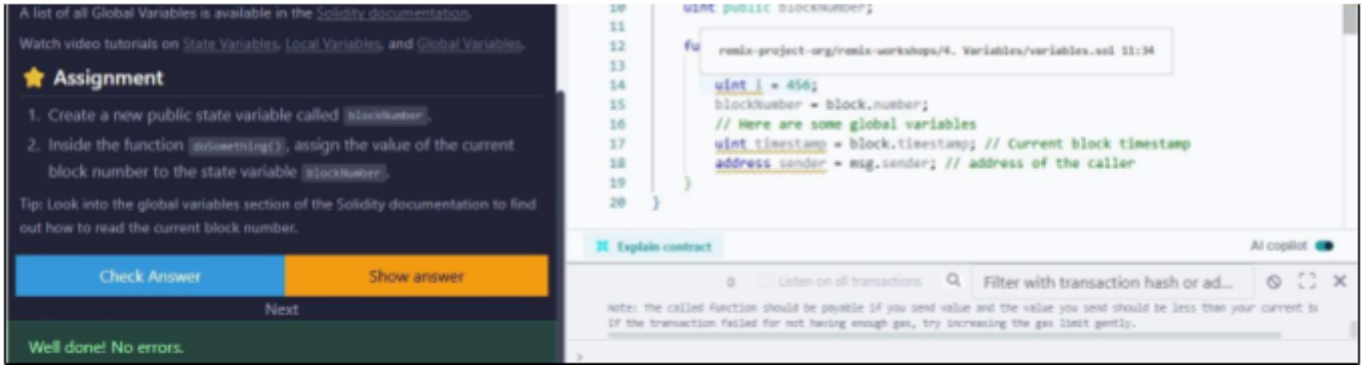
Tutorial - 2



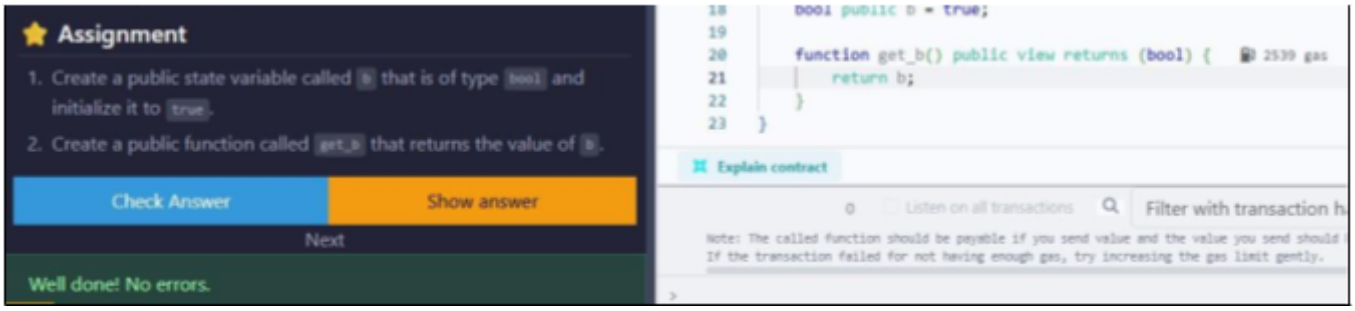
Tutorial - 3



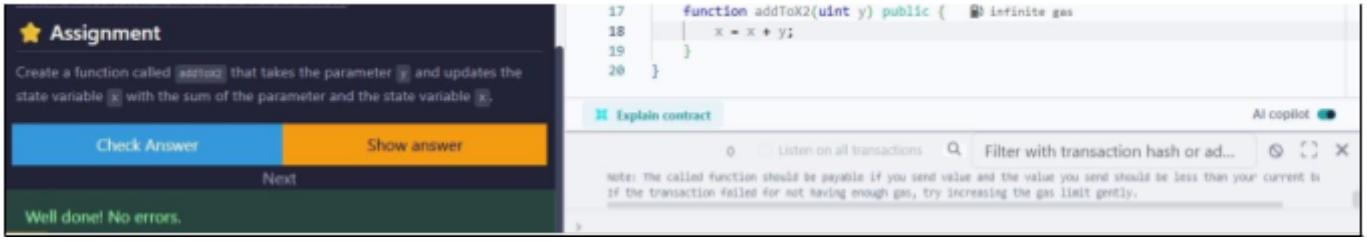
Tutorial - 4



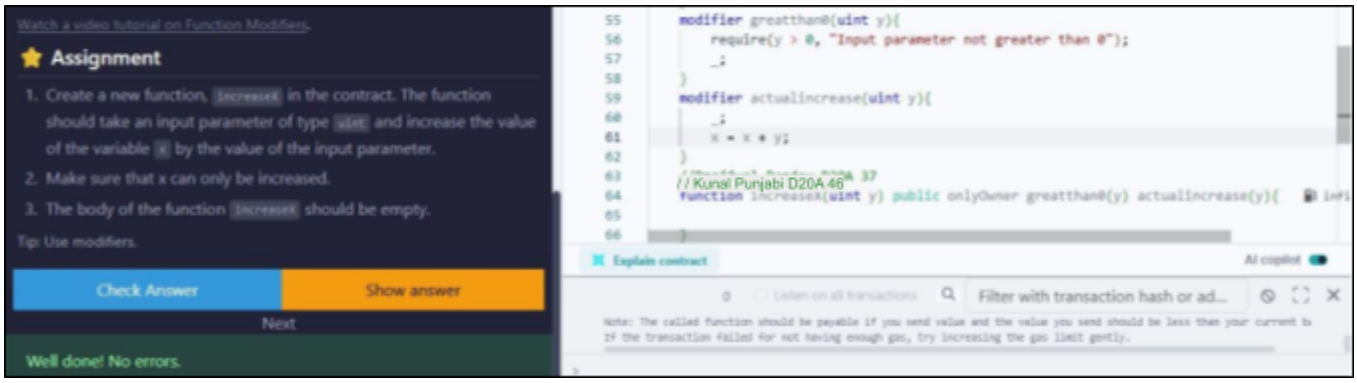
Tutorial - 5



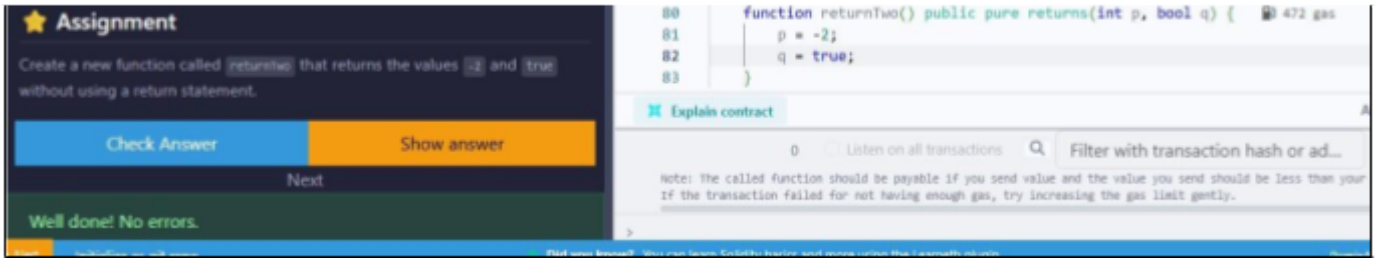
Tutorial - 6



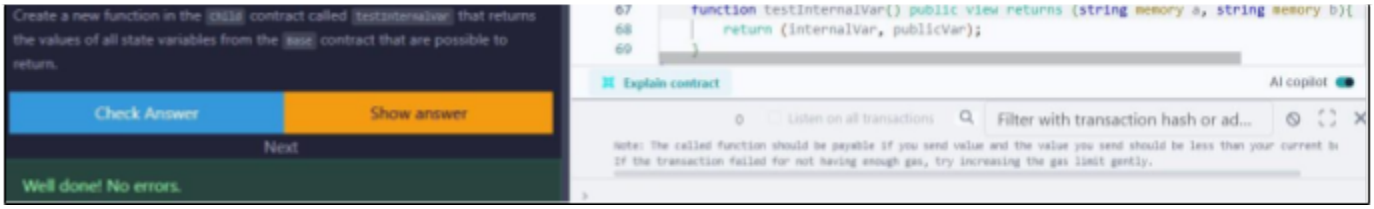
Tutorial - 7



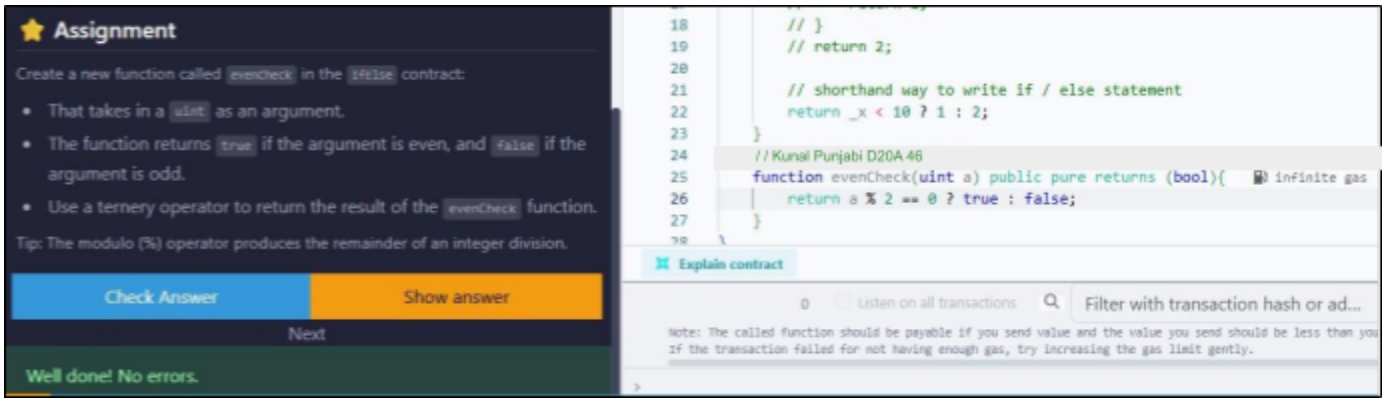
Tutorial - 8



Tutorial - 9



Tutorial - 10



Tutorial - 11

Tutorials list | Syllabus

7.2 Control Flow - Loops
11 / 19

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

1. Create a public `uint` state variable called count in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer | Show answer

Next

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Loop {
5   // Kunal Punjabi D20A 46
6      uint public count;
7      function loop() public {    infinite gas
8          // for loop
9          for (uint i = 0; i < 10; i++) {
10             if (i == 5) {
11                 // Skip to next iteration with continue
12                 continue;
13             }
14             if (i == 5) {
15                 // Exit loop with break
16                 break;
17             }
18             count++;
19         }
20
21         // while loop
```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or ad...

Note: The called function should be payable if you send value and the value you send should be less than your current b
If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 12

Tutorials list | Syllabus

8.1 Data Structures - Arrays
12 / 19

important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer | Show answer

Next

Well done! No errors.

```
7      uint[] public arr2 = [1, 2, 3];
8  // Kunal Punjabi D20A 46 D20A 37
9      uint[3] public arr3 = [0, 1, 2];
10     // Fixed sized array, all elements initialize to 0
11     uint[10] public myFixedSizeArr;
12
13     function get(uint i) public view returns (uint) {    infinite gas
14         return arr[i];
15     }
16
17     // Solidity can return the entire array.
18     // But this function should be avoided for
19     // arrays that can grow indefinitely in length.
20     function getArr() public view returns (uint[3] memory) {    infinite gas
21         return arr3;
22     }
23
24     function push(uint i) public {    46820 gas
25         // Append to array
26         // This will increase the array length by 1.
```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or ad...

Note: The called function should be payable if you send value and the value you send should be less than your current b
If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 13

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer | Show answer

Next

Well done! No errors.

```
    mapping(address => uint) public myMap;
// Kunal Punjabi D20A 46ey D20A 37
    mapping(address => uint) public balances;

    function get(address _addr) public view returns (uint) {    2895 gas
        // Mapping always returns a value.
        // If the value was never set, it will return the default value.
        return balances[_addr];
    }

    function set(address _addr) public {    25279 gas
        // Update the value at this address
        balances[_addr] = _addr.balance;
    }

    function remove(address _addr) public {    5588 gas
```

Explain contract

0 Listen on all transactions Filter with transaction hash or ad...

Note: The called function should be payable if you send value and the value you send should be less than yo
If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 14

Watch a video tutorial on Structs.

Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer | Show answer

Next

Well done! No errors.

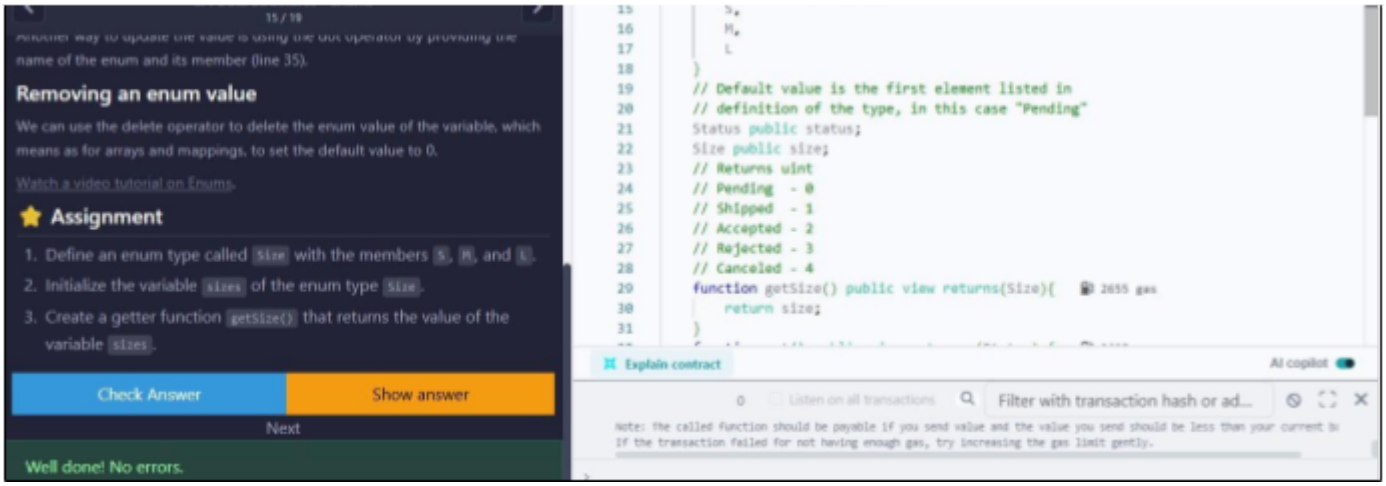
```
    }
// Kunal Punjabi D20A 46
    function remove(uint _index) public {    infinite gas
        delete todos[_index];
    }
```

Explain contract

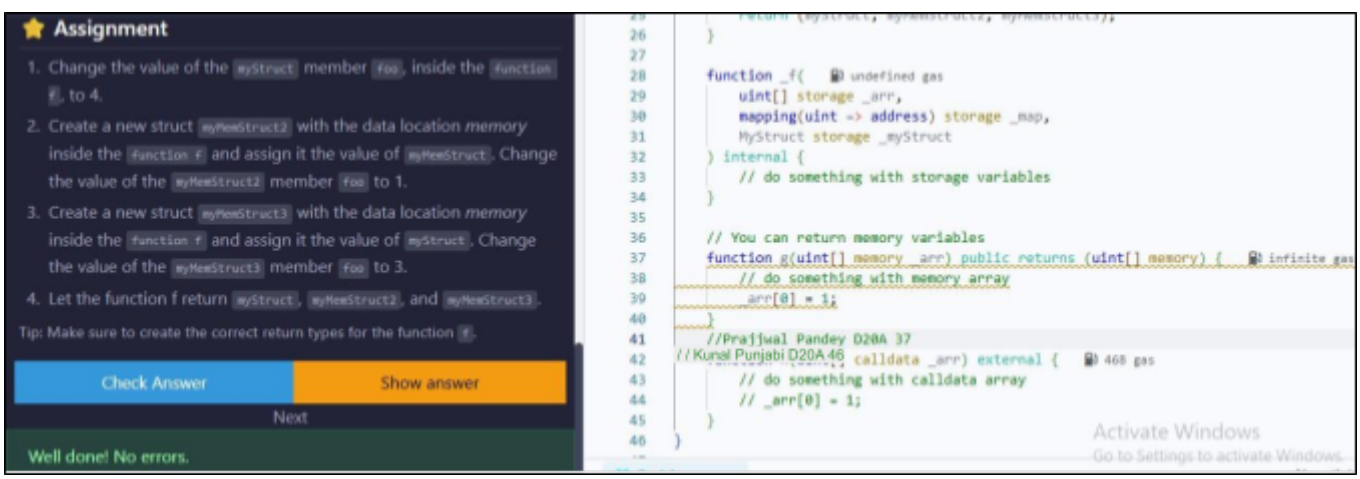
0 Listen on all transactions Filter with transactio

Note: The called function should be payable if you send value and the value you send sh
If the transaction failed for not having enough gas, try increasing the gas limit genti

Tutorial - 15



Tutorial - 16



Tutorial - 17

Tutorials list

Syllabus

10.1 Transactions - Ether and Wei

17 / 19

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000,000 (10^18) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public` `uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public` `bool` called `isOneGwei` and set it to the result of a comparison operation between 1 gwei and 10^9.

Tip: Look at how this is written for `gwei` and `ether` in the contract.

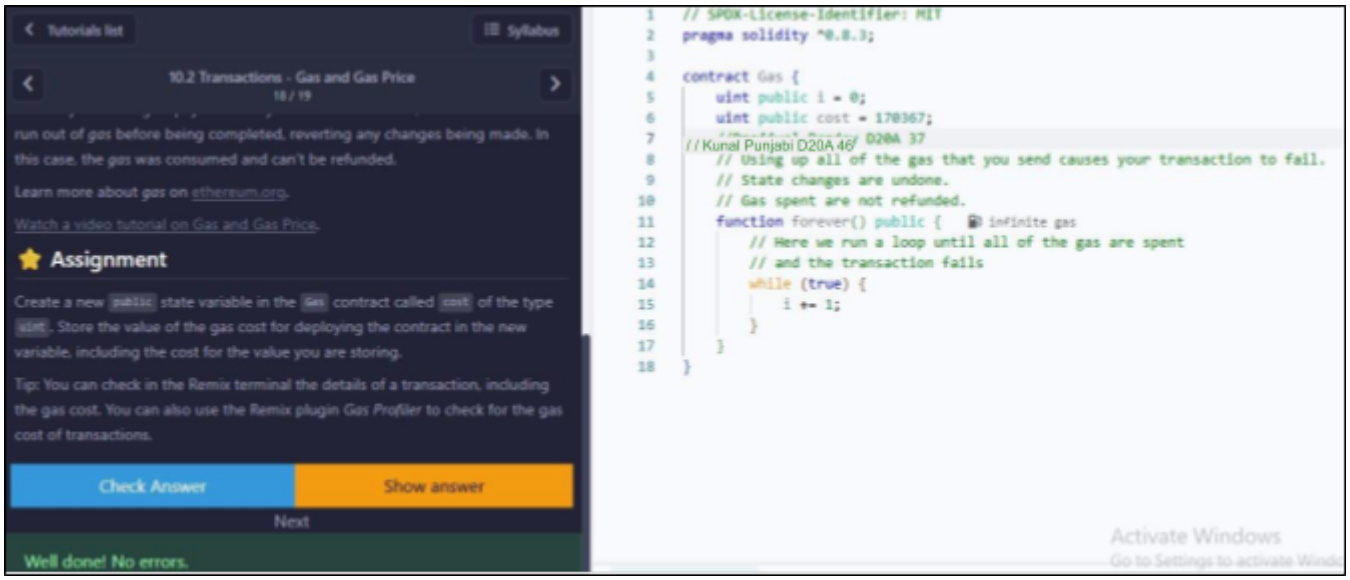
Check Answer | Show answer

Next

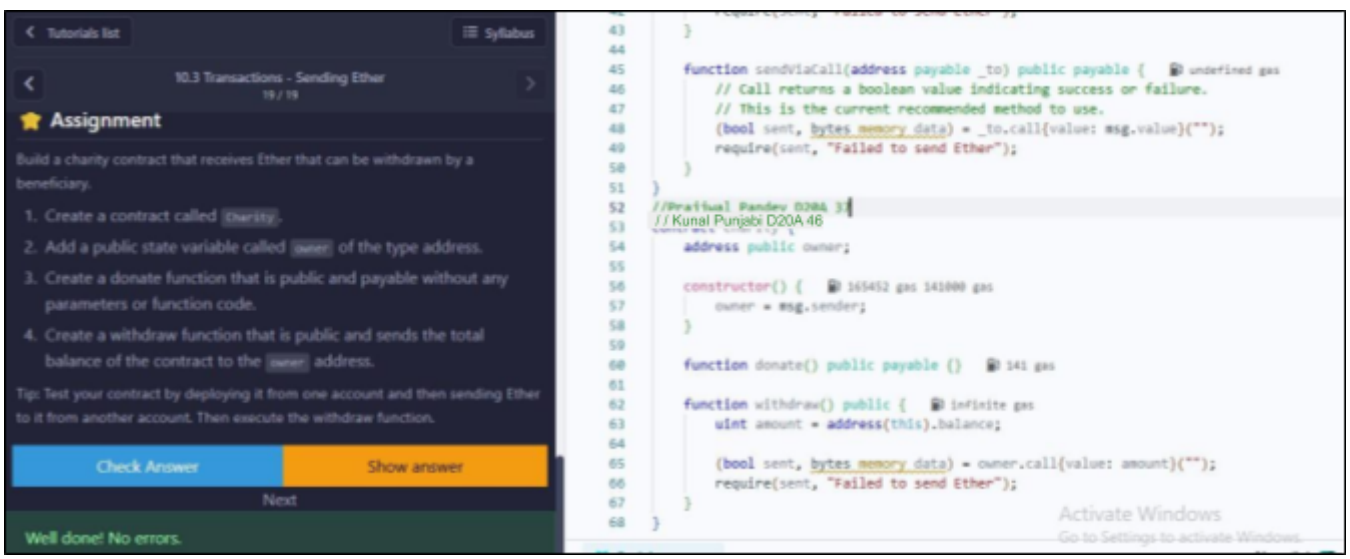
Well done! No errors.

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract EtherUnits {
5      uint public oneWei = 1 wei;
6      // 1 wei is equal to 1
7      bool public isOneWei = 1 wei == 1;
8
9      uint public oneEther = 1 ether;
10     // 1 ether is equal to 10^18 wei
11     bool public isOneEther = 1 ether == 1e18;
12
13     uint public oneGwei = 1 gwei;
14     // 1 ether is equal to 10^9 wei
15     bool public isOneGwei = 1 gwei == 1e9;
16 }
17 // Kunal Punjabi D20A 46
```

Tutorial - 18



Tutorial - 19



Conclusion: Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.